RAYCAST AD: THE PROMPT AND THE CODE

Inside: the actual scene structure, a prompt for Claude Code, selected code from my Remotion build, and the mistakes that changed the result. This is a method, not a copy of my video. Your assets and decisions will make your own result.

A question, a shortcut, eight real features, a 3D camera, and sound placed on the frame of each action.

Unofficial concept ad by Veronika. Not commissioned by, sponsored by, or affiliated with Raycast.

ONE KEY. EVERY TOOL.

The 22.7 second wide ad begins with “How many apps did you open today?” App icons fill the depth behind the question. A shortcut brings them into one search bar.

`0 to 100`	Question, 3D icon field, then Open / Search / Switch / Copy / Paste / Repeat.
`100 to 155`	Option + Space, the one-key moment, and a Raycast search bar.
`155 to 416`	Eight real features. Each card reveals its input and result in layers.
`416 to 482`	Three phone mockups with real iOS screens.
`482 to 556`	Extension icons fly into one panel.
`556 to 680`	Logo ribbon, wordmark, and Raycast's slogan.

A separate vertical “desk” version places the finished ad on a monitor and an edit timeline on a laptop. It uses a photographed workspace and perspective-mapped screens. This guide describes how to build that with your own photograph.

> **The rule that changed the reel:** use real product material. The first draft used invented interface windows and icons. The result was hard to understand.

START WITH THE BRAND

Study the official product site and press kit before asking Claude Code to animate anything.

1. Open raycast.com/press for logos, colors, app icons and product screenshots. The press page specifies red `#FF6363` and preferred dark background `#151515`.
2. Open raycast.com for actual feature cards, extension examples and interface language. Save the assets you choose in your own project folder, then map each asset path in the scene data.
3. Download Inter from its official site, or choose another font you have rights to use. Match the brand's weight and spacing. Check the license before publishing.
4. Choose your own licensed sound effects. Make folders for impacts, whooshes, clicks and rises. Put their paths in your own audio script. Use your own desk photograph if you want the vertical version.

> **Nothing to download from this guide:** no Raycast asset files, sound packs, SF Pro or Coolvetica font files, or desk photograph are included. Official pages can change. Recheck current brand assets and usage terms before making your version.

Keep a small asset manifest: scene, source URL, local file, crop coordinates, usage rights. For cards, crop the relevant UI from a real screenshot before rendering. A huge image in every frame slows Remotion down.

PASTE INTO CLAUDE CODE

CLAUDE CODE / START PROMPT

```
Make an unofficial premium concept ad for <BRAND> in Remotion,
1920 x 1080, 30 fps, with a separate vertical desk presentation.
First study the brand's official site and press kit. Make an asset
manifest with source URLs. Use only real product UI, icons and claims.
Ask me for my motion reference, licensed SFX and desk photo.
Write a frame-based scene plan before coding. Open with a question to
the viewer. Bring scattered app icons into one shortcut. Show eight
real features as cards: reveal input, then result. Use an inner camera
that follows the typing and an outer 3D camera that arcs around cards.
Then show phone screens, tools converging into one panel, and the logo.
Use CSS perspective and preserve-3d. Keep text sharp and readable.
At 120 BPM and 30 fps, a quarter-note beat is 15 frames. Place each
sound's peak on the action frame using my licensed SFX files. Leave
space for short voice lines at the beginning of scenes. No music.
Render the opening first, then the full ad. Review a frame sheet and
watch the motion and listen to the mix before calling it finished.
Label the result "Concept ad, not affiliated with <BRAND>."
```

Give Claude Code your chosen asset paths, frame plan and the exact crop of each real UI card. The prompt is a starting brief. It cannot reproduce my edit by itself.

> **Setup:** Install Node.js, create a Remotion project, put your own licensed assets in its `public/` folder, and paste this prompt into Claude Code in that project. Detailed setup will be in the paid guides.

MAKE THE FRAMES EXPLICIT

Selected, shortened code from `kit.tsx`, `scenes.tsx` and `Ad.tsx`. Replace every asset path with your own files.

KIT.TSX / PALETTE AND DIMENSIONS

```
export const W = 1920;
export const H = 1080;
export const RED = '#FF6363';
export const INK = '#151515';
```

SCENES.TSX / ONE TIMELINE IN FRAMES

```
export const T = {hook: 0, key: 100, calc: 155,
  trans: 192, file: 227, win: 260, shot: 292,
  emoji: 323, ai: 353, notes: 386,
  phones: 416, ext: 482, final: 556, end: 680};
const WORDS = [{t:'Open.',f:60},{t:'Search.',f:67},
  {t:'Switch.',f:74},{t:'Copy.',f:81},
  {t:'Paste.',f:88},{t:'Repeat.',f:95}];
```

The hook uses `perspective: 1100`. Each icon has its own `translateZ`; the distant icons soften, and icons under the question are dimmed for readability. The question sits on a plane that rotates from `rotateX(22deg)` toward zero.

AD.TSX / SCENE SELECTION AND CAMERA KICK

```
const kick = KICKS.reduce((s, k) =>
  s + (g >= k ? Math.exp(-(g-k)/5) : 0), 0);
let cur = SC[0];
SC.forEach(sc => { if (g >= sc.from) cur = sc; });
<AbsoluteFill style={{transform:
  `scale(${1 + Math.min(kick,1.4)*0.024})`}}>
  {cur.render(g - cur.from)}
</AbsoluteFill>
```

A ribbon crosses each scene boundary over 12 frames. The scene switches at the exact timeline frame while the ribbon masks the cut.

PRINT A REAL CARD

One source image becomes three reveals. The asset is never redrawn as a fake interface.

SCENES.TSX / FEATURE, SIMPLIFIED FROM THE BUILD

```
const type = p(f, tAt, tDur, eOut);
const res = p(f, tAt+tDur+1, ft.resDur ?? 8, eOut);
const layer = (key, clip) => (
  <div key={key} style={{position:'absolute',
    width:w, height:h, clipPath:clip}}>
    <Img src={ft.src} style={{position:'absolute',
      left:-cx*s, top:-cy*s, width:ft.iw*s,
      height:ft.ih*s, maxWidth:'none'}} />
  </div>
);
const layers = [
  layer('ribbon', `polygon(0 0, ${620*s}px 0,
    0 ${380*s}px)`),
  layer('input', `inset(${rt}px ${(1-type)*w}px
    ${h-re}px 0)`),
  layer('result', `inset(${re}px 0
    ${(1-res)*(h-re)}px 0)`)
];
```

The input reveal reduces the right inset, so it appears to type from left to right. The result rises from the lower crop. The input begins around feature frame 3; the result follows around frame 11.

SCENES.TSX / TWO CAMERAS

```
const tx = clampN(w/2-z*Tx, w-z*w, 0);
const ty = clampN(h/2-z*Ty, h-z*h, 0);
const camY = dir*lerp(-20, 9, arc);
const camZ = lerp(-380, 80, p(f,0,12,eExp));
// inner: translate(tx,ty) scale(z) inside card
// outer: perspective 2400, preserve-3d,
//        translateZ(camZ) rotateY(camY)
```

Clamp the inner camera so the screenshot never exposes an empty card edge. Neighboring cards sit behind it at negative Z with lower opacity.

LAND ON THE ACTION

My `make-audio2.py` loads my own licensed SFX, converts them to stereo at 48 kHz and places each effect by frame. Use your own pack names and paths.

MAKE-AUDIO2.PY / THE PLACEMENT IDEA

```
SR, FPS, FRAMES = 48000, 30, 680
def put(rel, frame, db, at_peak=True, end=False):
    a = load(rel)  # replace with your licensed file
    if end:
        i = int(frame/FPS*SR) - len(a)
    elif at_peak:
        i = int(frame/FPS*SR) - peak_of(a)
    else:
        i = int(frame/FPS*SR)
    a *= 10 ** (db/20)
    mix[i:i+len(a)] += a
```

`at_peak` aligns the loudest moment with the visible action. `end` lets a rise finish exactly on a cut. For the final mix, layer an impact, a whoosh and a click only where the picture needs them. The source mix also uses light panning, short reverb tails and loudness normalization. There is no music.

The voice was generated separately in ElevenLabs, then split by pauses and aligned to the starts of scenes. The very fast Open-to-Repeat montage stays unvoiced. Feature voice lines are one word each. Duck effects under speech and listen to the final mix; a contact sheet cannot tell you whether a voice is audible.

PUT THE AD ON A DESK

Use your own desk photograph. Mask the monitor, mark its four corners, and perspective-map the video into that quadrilateral. Add your own laptop screen with scene thumbnails, timeline tracks and a playhead. A softened copy of the video can cast colored light over the wall and desk. Use slow camera drift, not text shake. The vertical presentation in this project lives in `Desk.tsx`; its photo, mask and font files are not supplied here.

What went wrong

- Invented product windows made the first version unclear. Use real screenshots and accurate feature names.
- White type on Raycast red had weak contrast. Dark type on red read better.
- White flashes on dark frames made a gray veil. Use a faint warm flash.
- The first frame looked empty. Start some icons immediately.
- A glowing line through the entire ad competed with the product. The final edit removed it.
- A serif headline did not fit the rest of the design. The final ad uses Inter.
- Long voice phrases could not keep up with short feature scenes. Use a word at the start of each scene.
- Large PNGs slowed rendering. Crop before using them in frame-by-frame composition.

MAKE YOUR VERSION.

This guide gives you the method, prompt and selected code. Your video will look different. Reel Studio contains the exact templates behind my videos, so you can build them one to one. Join the early list on my site.